

WasmAndroid: A Cross-Platform Runtime for Native Programming Languages on Android (WIP Paper)

Elliott Wen
The University of Auckland
New Zealand
jq.elliott.wen@gmail.com

Gerald Weber
The University of Auckland
New Zealand
g.weber@aucklanduni.ac.nz

Suranga Nanayakkara
The University of Auckland
New Zealand
suranga@ahlab.org

Abstract

Open-source hardware such as RISC-V has been gaining substantial momentum. Recently, they have begun to embrace Google's Android operating system to leverage its software ecosystem. Despite the encouraging progress, a challenging issue arises: a majority of Android applications are written in native languages and need to be recompiled to target new hardware platforms. Unfortunately, this recompilation process is not scalable because of the explosion of new hardware platforms. To address this issue, we present WasmAndroid, a high-performance cross-platform runtime for native programming languages on Android. WasmAndroid only requires developers to compile their source code to WebAssembly, an efficient and portable bytecode format that can be executed everywhere without additional reconfiguration. WasmAndroid can also trans-pile existing application binaries to WebAssembly when source code is not available. WebAssembly's language model is very different from C/C++ and this mismatch leads to many unique implementation challenges. In this paper, we provide workable solutions and conduct a preliminary system evaluation. We show that WasmAndroid provides acceptable performance to execute native applications in a cross-platform manner.

CCS Concepts: • Computer systems organization → Architectures.

Keywords: WebAssembly RISC-V Mobile Portability

ACM Reference Format:

Elliott Wen, Gerald Weber, and Suranga Nanayakkara. 2021. WasmAndroid: A Cross-Platform Runtime for Native Programming Languages on Android (WIP Paper). In *Proceedings of the 22nd ACM SIGPLAN/SIGBED International Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES '21)*, June 22, 2021, Virtual,

Canada. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3461648.3463849>

1 Introduction

Recently, the open-source hardware movement has established a solid foothold. Exemplary leaders include RISC-V [1], OpenRISC [2], and Microwatt [3], which are all license-free Instruction Set Architectures (ISA). They have a variety of advantages including openness, modularity, extensibility and stability. Thanks to these benefits, They are ushering in a new era of silicon design and processor innovation. Recently, they have begun to embrace Google's Android operating system [4]. Android is open-source and widely used in embedded platforms and industry sectors. The porting of Android enables the open-source hardware to leverage the mature Android developer ecosystem. It also provides a potential escape route for mobile device vendors, who are wary of proprietary licensing pitfalls.

However, to fully embrace open-source hardware platforms, a huge challenge yet awaits developers: a considerable number of Android applications are written in native languages (C/C++) and need to be recompiled to target new ISAs [5]. This process can be very labor-intensive thanks to the explosion of new open ISAs. What makes the process really not scalable is that open ISAs typically allow chip designers to customize instruction extensions. For example, they can decide whether to include FPU (floating point units), SIMD (Single Instruction Multiple Data), or other coprocessors to trade off hardware resources and computing performance. For each instruction configuration, developers are also forced to prepare one corresponding native binary.

To address these issues, we present WasmAndroid, a high-performance cross-platform runtime for native programming languages on the Android OS. With WasmAndroid, application developers do not need to generate native binaries for every CPU architecture. Instead, they can compile source codes into WebAssembly, a bytecode format recently invented by mainstream browser vendors [6]. WebAssembly is entirely agnostic to hardware architectures and operating systems. Hence, a program can be compiled into WebAssembly once and executed everywhere without any additional reconfiguration. Unlike other high-level languages bytecode, WebAssembly instruction set is closely related to modern processors' instruction sets. Therefore, a WebAssembly binary

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

LCTES '21, June 22, 2021, Virtual, Canada

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8472-8/21/06...\$15.00

<https://doi.org/10.1145/3461648.3463849>

can be efficiently translated into native CPU instructions, delivering an execution speed as fast as native. It is worthwhile to mention that WasmAndroid can also trans-pile many existing application binaries, which presumably target the ARM architecture, to WebAssembly. This feature could be useful when we do not have access to an application's source code.

However, incorporating WebAssembly with Android entails many technical challenges. Firstly, WebAssembly programs expect a linear memory model, where memory content is laid out contagiously starting from address 0. This model is very different from Android's memory model, where an Android application is usually allocated in a random high memory address by default. Secondly, WebAssembly discarded the support to raw function pointers in C/C++. Instead, it uses an array-like structure (function table) to store functions' memory addresses. To reference or invoke one of these functions, WebAssembly needs to use its array index. This feature can be problematic because the Android Java runtime frequently passes raw function pointers to native code. Additionally, WebAssembly does not have built-in instructions for system calls. The only workaround is through external functions exposed to WebAssembly via the function table. By exposing fewer native system interfaces, we can reduce the attack surface of native code. However, some applications may also stop working due to the inaccessibility to necessary system calls. It is tricky to strike a balance between functional correctness and security for different applications.

In this paper, we present practical solutions to cope with the above challenges. To address the impedance mismatch in memory models, our WebAssembly runtime can generate necessary instructions that map WebAssembly memory addresses to the Android memory space. This mapping overhead can be avoided with slight modifications to the WebAssembly toolchain. In addition, our WebAssembly runtime can identify functions that expect function pointers and conduct necessary transformations on their parameters. Our system also allows users to specify what native interfaces to expose to WebAssembly using Lua, a simple-yet-expressive programming language.

We implemented a prototype of WasmAndroid for Android 10 running on a RISC-V emulator. We then evaluated the prototype using SPEC CPU 2006 benchmark suites. The experiment results show that WasmAndroid provides acceptable performance to execute native applications in a cross-platform manner.

2 Background

Android & Open Hardware. In Android, developers can use native languages to implement computationally intensive components or to impede reverse engineering. To facilitate this process, Android provides a native development toolkit (NDK) [7], which are essentially C/C++ compilers

and platform libraries. The compiled binaries then can be dynamically loaded and invoked by Java code using Java Native Interface (JNI). One downside of NDK is that the binaries are not portable across different CPU architectures. This was only a minor concern because there were only two dominant mobile CPU vendors X64 and ARMV8a. Developers could just bundle two types of binaries in their application installation files. Recently, the open-source hardware movement is gaining enormous momentum. The explosion of open processors [1–3] extensively aggravates the portability issue of NDK.

WebAssembly. WebAssembly is a low-level bytecode format originally designed for web browsing. It is independent of programming languages, hardware architectures, and operating systems. Recently, a growing number of compilers (e.g., C/C++ [8], Rust [9] and Javascript [10]) has begun to generate WebAssembly bytecode. The generated bytecode can be directly executed in mainstream browsers. Thanks to WebAssembly, many novel web applications such as 3D gaming [11], audio and video processing [12], and virtualization [13] have been invented. More recently, researchers are exploring WebAssembly's potential beyond the Web, especially on IoT and Fog Computing. Commercial-grade Cloudflare [14] and a recent research work [15] exploit secure features of WebAssembly to enable sandbox execution of serverless WebAssembly functions. Jacobsson et al., [16] and Wen et al., [17] exploit WebAssembly's portability to run the same WebAssembly binary across micro-controllers with different hardware architectures. These works motivate us to build a cross-platform runtime for native programming languages on Android using WebAssembly.

3 System Architecture

Figure 1 demonstrates the main work flow of WasmAndroid. Given an Android application's native source code, developers can compile it into WebAssembly bytecode using a WebAssembly Clang toolchain (wasm64). Alternatively, given an existing binary file, developers may transpile it to WebAssembly directly. Currently, our transpiler is only able to process ARM binaries. The support to x64 binaries is still experimental because x64 is a CISC instruction set, which is troubled by the infamous code discovery problem. The generated WebAssembly bytecode will be embedded in an android application package (APK) and distributed to mobile devices with different CPU architectures. This compilation process needs only to be carried out once because WebAssembly bytecode is portable.

To install such an APK, a patched Android device will first extract the WebAssembly bytecode. The bytecode is hardware-independent and not directly executable by physical CPUs. It has to be fed into a WebAssembly runtime to generate native CPU instructions before execution. A WebAssembly runtime can be implemented using an interpreter,

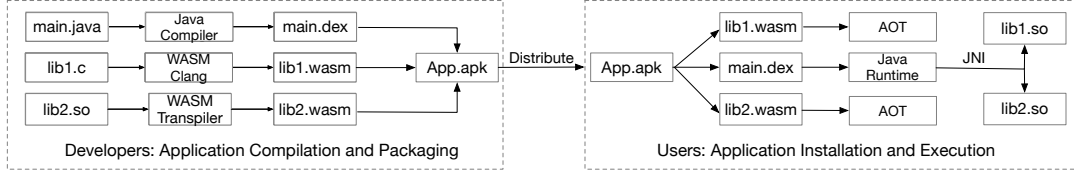


Figure 1. WasmAndroid enables native code to be compiled once and execute everywhere.

which translates and executes one WebAssembly instruction at a time. A runtime can also be implemented in an ahead-of-time (AOT) compilation manner, where a WebAssembly program is transformed into native machine code at once before it runs. WasmAndroid provides both of the implementations. By default, WasmAndroid adopts the AOT compiler, since it can perform complex code optimizations to generate highly efficient binary objects. The resulting objects can be dynamically invoked by the Android Java runtime using Java Native Interface (JNI).

4 AOT Compilation for WebAssembly

The core of WasmAndroid is the AOT WebAssembly compiler. In this section, we will illustrate the implementation challenges and solutions.

Instruction Translation. We construct an AOT compiler for WebAssembly with the help of LLVM compiler infrastructure [18]. In detail, we first implement a lexer and a parser for WebAssembly bytecode using Flex and Bison [19]. These tools enable us to iterate each WebAssembly instruction and convert it into LLVM intermediate representation (LLVM IR). The conversion is generally not difficult since both WebAssembly and LLVM IR are low-level instructions and many of them are semantically similar. One difficulty we encounter is that LLVM IRs are register-based (i.e., operands and results are stored in registers) while WebAssembly instructions are stack-based (i.e., pops operands from and pushes results onto a stack). To conduct the conversion, we need to maintain a stack structure for the mapping between the WebAssembly operands and LLVM registers. It is worth noting that the stack structure only exists during compilation because WebAssembly operands’ locations can always be statically determined due to WebAssembly’s structured control flow. [6]. The direct conversion result is generally suboptimal and can be extensively improved using LLVM optimization passes (e.g., constant elimination and loop unrolling). The optimized IRs are then sent to LLVM backends to generate native objects for the target CPU architecture. From the Android Java runtime’s perspective, the resulting objects behave the same as the native libraries generated using Android’s native development kit (NDK).

Memory Mapping. Each WebAssembly application expects one linear memory, which is byte-addressable and spanning from address 0 to mutable memory size. The memory content can be accessed by memory instructions ‘load’

and ‘store’ with an index parameter. For example, a WebAssembly instruction ‘i32.load 100’ will load a 32-bit integer located in memory locations 100-103. This is very different from the memory model of real-world OSs, where applications are randomly allocated in a high virtual memory address space (e.g., 0xffffffff80000000). To bridge the difference, our compiler generates necessary instructions to map each WebAssembly memory address to a valid memory address in the OS. In detail, our AOT compiler first generates a program initialization routine (i.e., ctor function) to allocate a contiguous memory chunk from the underlying OS. The region’s address is then recorded in a global variable *linear_mem*. Afterward, the compiler can offset each WebAssembly memory operation’s parameter with the value of *linear_mem*. For instance, the WebAssembly instruction ‘i32.load 100’ will be translated into LLVM IR ‘%1 = load i32, i32* @linearmem; %transformed_addr = 100 + %1; %result = load i32, i32* %transformed_addr’. We can further inject bounds-checking instructions on ‘%transformed_addr’ to ensure that all memory operations are well-defined. If out-of-bound access is detected, we can terminate the application immediately to ensure system safety.

However, such a transformation inevitably incurs a significant performance penalty because it inserts more than ten instructions for each memory operation. This paper provides a more efficient approach, which can completely avoid the memory address transformation and bounds checking. The key idea is to use the Linux system call ‘mmap’ to allocate a memory region in a low virtual memory address space. Ideally, the region can start at address 0 and has the same size as the WebAssembly linear memory. In this case, address transformation is not required as all WebAssembly memory addresses are now valid and accessible. Bounds checking is also not required since visiting a non-allocated virtual address will always trigger a segment fault in Linux. Unfortunately, in practice, the system call ‘mmap’ generally disallows memory allocation starting from address 0 to prevent NULL pointer dereference exploits in kernel space. A simple workaround is to adjust the kernel parameter ‘mmap_min_addr’, which specifies the minimum virtual address (4096 by default) that a process can allocate. This, however, may extensively increase the security implications. In this paper, we bypass the ‘mmap’ restriction safely by applying minor modifications on the Clang toolchain. The patched toolchain will generate WebAssembly programs whose linear memory is filled

with garbage data in locations 0-4095. The garbage data will never be referenced, we thus do not need to load them to the main memory. Furthermore, all the WebAssembly memory instructions will have an address parameter larger than 4095, implying that we only need to 'mmap' a memory region starting from address 4096.

Function Pointer Transformation. In C/C++, a function pointer is represented by a function's memory address. Though powerful, function pointers are culprits of many common security vulnerabilities such as stack smashing and control-flow hijacking. WebAssembly discarded the support to raw function pointers. Instead, WebAssembly introduces a primitive called function table to store functions' memory addresses. Every function in the table is referenced by a unique index, which can be stored in the linear memory and passed around to simulate traditional function pointers. To invoke such a function, WebAssembly needs to use the instruction 'call_indirect' and supply its index. The index will be safety bounds checked against the table before indexing and jumping to the function address.

This security feature leads to a unique challenge when we inter-operate the Android's Java bytecode and native WebAssembly bytecode. Specifically, each native method invoked by Java will receive a 'JNIEnv' as the first argument. 'JNIEnv' is essentially a C structure containing raw function pointers necessary to interact with the Android Java runtime and Java objects (e.g., Java/native array conversion, objects initialization, and throwing exceptions). Unfortunately, these raw function pointers are not invocable by WebAssembly bytecode before transforming them into function table indexes. To achieve that, our AOT compiler generates a wrapper function for each native method. The wrapper first iterates every raw function pointer in the incoming 'JNIEnv' structure. If a raw function pointer does not exist in the function table, it will be appended to the table. After that, the wrapper duplicates a 'JNIEnv' structure and replaces every raw function pointer inside with the corresponding function index. The transformed 'JNIEnv' will then be passed to the original native method.

Note that wrappers are only required for methods exposed to Java (i.e., public JNI method). However, it sometimes can be tricky to identify those functions depending on how they are linked. Developers usually use automatic JNI linking, where they are required to follow a specific naming convention for their public methods. In this case, we can quickly identify the exported functions by checking their names. Occasionally, developers may also use an API called 'RegisterNative' to register public methods with arbitrary names in runtime. We are still experimenting with different strategies to cope with this setting. We could conduct statics analysis in the clang toolchain and embed the result in the WebAssembly bytecode. We may also detect public methods in runtime by hooking the 'RegisterNative' API and generate the necessary wrappers on the fly.

System Call Protection. WebAssembly has no built-in privileged instructions for system calls or hardware I/O. Hence, WebAssembly bytecode cannot alone express and perform any security-sensitive behaviors. The only workaround is to register external system interfaces in WebAssembly's function table. This feature offers a software fault isolation mechanism to reduce the attack surface of native code. For example, if the native code is designed only to conduct mathematical computation, we do not need to expose any external methods. Similarly, if the native code is not expected to access the Internet, we do not have to expose socket system calls.

Since every native application may expect different system calls, it would be beneficial to allow application developers to specify a minimum set of required system calls. To achieve that, WasmAndroid harnesses the simplicity of Lua, a dynamically typed and memory-safe scripting language. Specifically, we allow developers to embed an optional Lua script as auxiliary information in the WebAssembly bytecode. In the script, developers need to implement a set of Linux system call wrappers. A wrapper will be invoked whenever WebAssembly requests the corresponding system call. The wrapper can return 'ENOSYS', meaning the system call is not restricted. In contrast, the wrapper can invoke the original system call and pass the parameters as is. This wrapper then becomes a proxy to the system call. A more advanced usage is to examine the system call parameters and filter them if an attempt to exploit kernel vulnerabilities is detected. This feature can enable the live patching of kernel-level vulnerabilities.

5 Preliminary Experiments

We conduct a preliminary performance evaluation of WasmAndroid. Specifically, we selected several benchmark executables from an established CPU benchmark suite *SPEC CPU 2006*. WasmAndroid in the AOT mode performs on average $1.34\times$ slower than the baseline for all benchmarks. The performance is arguably acceptable for many applications that are not performance-critical. We plan to conduct more thorough experiments in the future.

6 Conclusion

This paper presents WasmAndroid, a performant and portable runtime for native programming languages on Android. With WasmAndroid, developers can compile their native source code or transpile existing binaries to WebAssembly and execute it on different hardware platforms without additional reconfiguration. WebAssembly possesses a language model very different from C/C++ and this mismatch leads to many unique implementation challenges. In this paper, we present practical solutions and conduct a preliminary system evaluation. Our results show that WasmAndroid achieves comparable performance to their native counterparts.

References

- [1] A. S. Waterman, "Design of the risc-v instruction set architecture," Ph.D. dissertation, UC Berkeley, 2016.
- [2] J. Tandon, "The openrisc processor: open hardware and linux," *Linux Journal*, vol. 2011, no. 212, p. 6, 2011.
- [3] "Power instruction set architecture (isa)." [Online]. Available: <https://openpowerfoundation.org/tag/microwatt/>
- [4] "Android has been ported to a risc-v board." [Online]. Available: <https://www.xda-developers.com/android-risc-v-port/>
- [5] F. Wei, X. Lin, X. Ou, T. Chen, and X. Zhang, "Jn-saf: Precise and efficient ndk/jni-aware inter-language static analysis framework for security vetting of android applications with native code," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1137–1150.
- [6] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, "Bringing the web up to speed with webassembly," in *ACM SIGPLAN Notices*, vol. 52, no. 6. ACM, 2017, pp. 185–200.
- [7] S. Ratabouil, *Android NDK: beginner's guide*. Packt Publishing Ltd, 2015.
- [8] A. Zakai, "Emscripten: an llvm-to-javascript compiler," in *Proceedings of the ACM international conference companion on Object oriented programming systems languages and applications companion*. ACM, 2011, pp. 301–312.
- [9] N. D. Matsakis and F. S. Klock II, "The rust language," in *ACM SIGAda Ada Letters*, vol. 34, no. 3. ACM, 2014, pp. 103–104.
- [10] M. Reiser and L. Bläser, "Accelerate javascript applications by cross-compiling to webassembly," in *Proceedings of the 9th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages*. ACM, 2017, pp. 10–17.
- [11] A. Zakai, "Fast physics on the web using c++, javascript, and emscripten," *Computing in Science & Engineering*, vol. 20, no. 1, pp. 11–19, 2018.
- [12] S. Letz, Y. Orlarey, and D. Fober, "Compiling faust audio dsp code to webassembly," 2017.
- [13] E. Wen, J. Warren, and G. Weber, "Browservm: Running unmodified operating systems and applications in browsers," in *2020 IEEE International Conference on Web Services (ICWS)*. IEEE, 2020, pp. 473–480.
- [14] K. T. Varda, Z. A. Bloom, M. P. Majkowski, I. Stepanyan, K. Kloepper, D. O. Knecht, J. Graham-Cumming, and D. Grant, "Cloud computing platform that executes third-party code in a distributed cloud computing network," Jun. 25 2019, uS Patent 10,331,462.
- [15] A. Hall and U. Ramachandran, "An execution model for serverless functions at the edge," in *Proceedings of the International Conference on Internet of Things Design and Implementation*. ACM, 2019, pp. 225–236.
- [16] M. Jacobsson and J. Wåhlén, "Virtual machine execution for wearables based on webassembly," in *13th EAI International Conference on Body Area Networks (BodyNets 2018)*, 2018.
- [17] E. Wen and G. Weber, "Wasmachine: Bring iot up to speed with a webassembly os," in *2020 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE, 2020, pp. 1–4.
- [18] C. Lattner, "Llvm and clang: Next generation compiler technology," in *The BSD conference*, vol. 5, 2008.
- [19] J. Levine, *Flex & Bison: Text Processing Tools*. "O'Reilly Media, Inc.", 2009.